CHAPTER 16

Design Google Drive

Cloud file storage and sync across devices with sharing, versioning and efficient transfers.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a cloud file storage and synchronisation service like Google Drive / Dropbox / OneDrive: users **upload and download files, sync across multiple devices, view revision history, share files, and receive notifications** on changes.

Why interviewers ask it: Covers block-level storage, deduplication, delta sync, metadata consistency, and notification/long-polling.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Important features?	Upload/download, sync, notifications, file revisions, sharing.
Mobile / web?	Both.
File types / size?	Any type; max 10 GB.
Encryption?	Yes, at rest.
Users?	50 M signed-up, 10 M DAU, 10 GB free space each.

3. Requirements

Functional

- Upload and download files.
- Sync across devices.
- See revisions.
- Share files.
- Notify on edit/delete/share.

Non-functional

- **Reliability** - data loss is unacceptable.
- Fast sync (low bandwidth use).
- Scalable.
- High availability (work through server failures/slowness).

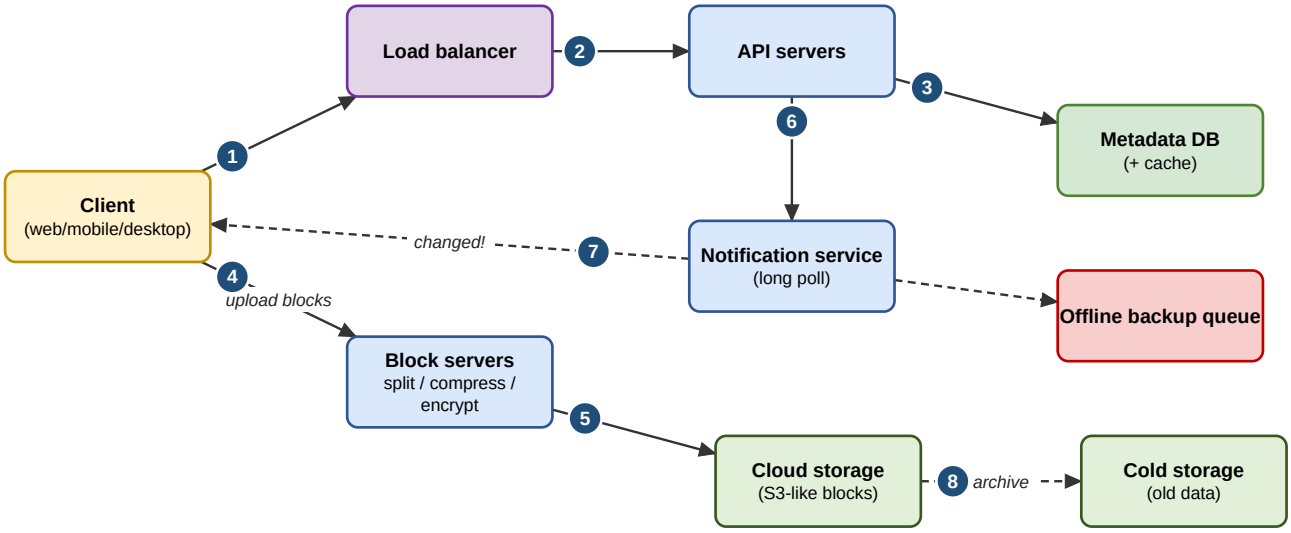
4. Back-of-the-envelope estimation

- 50 M users x 10 GB = **500 PB** allocated. 10 M DAU uploading 2 files/day x 500 KB = 10 TB/day.
- Upload QPS = 10 M x 2 / 86,400 = about 240 QPS, peak about 480.
- Block size 4 MB: 10 GB file = 2,500 blocks.

5. API / data model

```
POST /files/upload?uploadType=resumable   (chunked)
GET  /files/list_revisions?path=...&limit=20
GET /files/download?path=...
POST /share (file, user, permission)
```

6. High-level design



Step	What happens
1	The client first sends the file's **metadata** (name, size, hash list) through the load balancer to the API servers.
2	API servers create the file record with status `pending`.
3	The record is stored in the **metadata DB** (relational, strongly consistent) so every client sees the same file tree.
4	The client splits the file into **~4 MB blocks** and uploads them in parallel to **block servers**, which compress and encrypt each block.
5	Blocks are written to **cloud storage** (S3-like), identified by their hash; identical blocks are **deduplicated**.
6	On completion the file status becomes `uploaded` and the **notification service** is told.
7	Other devices are notified (long poll) that something changed; they fetch the metadata and download only the **changed blocks** (delta sync).
8	Old or rarely accessed blocks and versions move to **cold storage** to save cost.

Request flow

- **Upload:** client sends file metadata to API (status 'pending') and uploads **blocks** to block servers in parallel; block servers compress/encrypt and store in object storage; completion callback updates metadata to 'uploaded' and notifies other clients.
- **Sync/download:** notification service tells other devices something changed (long polling); they fetch updated metadata, then download only the **changed blocks** from block servers and reassemble the file.

7. Deep dive

Block servers and block-level storage

- Split files into blocks (Dropbox: up to 4 MB), each with a hash. Per-block compression and encryption. Store only blocks, then reassemble via a block list in metadata.

Delta sync

- When a file changes, upload only modified blocks rather than the whole file - saves bandwidth and time (like rsync).

Deduplication and compression

- Eliminate redundant blocks at account level by comparing hashes (content-addressed storage). Choose compression by file type (gzip/bzip2 for text; others for images/video).

Strong consistency for metadata

- Clients must see the same file view: caches/replicas with async replication are risky. Use a **relational DB** (ACID) for metadata (users, files, blocks, versions, devices) or invalidate caches on writes. NoSQL generally lacks ACID.
- Schema sketch: `user`, `device`, `namespace`, `file(id, name, latest_version, checksum...)`, `file_version` (read-only, immutable), `block(hash, order)`.

Notification service

- Keeps clients' view consistent: **long polling** (Dropbox) is used rather than WebSocket because communication is mostly server -> client, infrequent and no bursts.
- Notified clients fetch the latest metadata and changes.

Conflict resolution

- If two users edit simultaneously, first to be processed wins; the second gets a **conflict** copy: user can merge or keep both versions (e.g. `file (conflicted copy)`).

Save storage cost

- Limit and prune versions (cap count; keep fewer old ones); keep only valuable revisions; **cold storage** (S3 Glacier) for rarely used data; dedupe blocks; erasure coding instead of 3x replication.

Failure handling

- LB failure: secondary takes over (heartbeat). Block server failure: others pick up pending jobs. Cloud storage failure: replicate across regions. API server failure: stateless, LB reroutes. Metadata DB: master-slave failover. Notification server failure: clients reconnect (reconnect storms are a risk - slow ramp).

Sharing and permissions

- ACL per file/folder (viewer/editor/owner); share links with unguessable tokens; permission checks at the metadata layer on every access.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Sync unit	Whole file / block-level delta	Block-level delta for large, frequently changed files.
Metadata store	Relational (ACID) / NoSQL	Relational/strongly consistent for correctness of file trees and versions.
Notification	Long poll / WebSocket	Long poll: simple, enough for low-frequency server pushes.
Durability	3x replication / erasure coding	Replication for hot data; erasure coding for cold data.

9. Scaling, reliability, failure modes

- Shard metadata by user/namespace; blocks sharded by content hash in object store.
- Direct-to-storage uploads with pre-signed URLs to offload app servers.
- 11 nines durability through multi-AZ replication and checksums with periodic scrubbing.

10. Likely follow-up questions

- How do you handle offline edits and conflict resolution?
- How would you implement the real-time collaborative editing (Docs)? (OT / CRDT)
- How do you dedupe across users without leaking information about others' files?
- How do you handle very large files (resumable, multipart)?

11. Common mistakes

- Uploading/downloading entire files on every edit.
- Eventual-consistency metadata leading to users seeing different trees.
- Not covering durability (backups, replication, checksums).

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"The features: upload and download files, sync across devices, file revisions, sharing, and notifications. Any file type up to 10 GB, web and mobile, encrypted at rest, 10 M DAU and 10 GB free per user - that is about 500 PB allocated. Reliability is the top requirement because losing data is unacceptable."
2. High-level design	10 min	"Clients talk to API servers for metadata, which sits in a strongly consistent relational DB. File content is split into blocks and sent to block servers, which compress and encrypt and store them in cloud storage. A notification service tells other devices to sync. Old data moves to cold storage."
3. Deep dive	25 min	"Blocks are about 4 MB with a hash each, enabling delta sync, where only changed blocks are uploaded, and deduplication across the account. I use long polling for notifications since traffic is server-to-client and infrequent. Metadata needs strong consistency, so a relational DB. Conflicts: the first processed version wins and the second becomes a conflicted copy. To save cost: cap versions, cold storage, and erasure coding."
4. Wrap up	4 min	"Failure handling: LB pair, stateless API servers, replicated metadata with failover, multi-region storage. Extensions: real-time collaborative editing with OT or CRDTs, resumable large uploads and sharing permissions."

13. 60-second recap / practice checklist

- Blocks (4 MB) + block servers (compress/encrypt/dedupe) + object storage; metadata in a strongly-consistent DB.
- Delta sync + long-poll notifications keep devices in sync efficiently.
- Version history, conflict copies, cold storage for cost, replication for reliability.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Dropbox: Streaming file synchronization
• Dropbox: Scaling to exabytes (Magic Pocket)
• GeeksforGeeks: Design Dropbox